```
function HelperEncryption() %Since Part F is the main function, we put it
% at the beginning to call the other functions in order
clc;
clear;

%Ask the user to enter a message
text = input("please enter a plain text: ",'s');

%Go through the text and save the positions of any spaces, and skip the
%spaces and only add the other characters to a new string.
[nospace,spaceindex] = blankout(text);

%lowercase() function converts all uppercase letters into lowercase, and
%save the original positions of the uppercase letters. So in this case,
%this line convert the text to lowercase and save where the capital letters
%were.
[lower,capitalindex] = lowercase(nospace);

%Add the saved space positions to the beginning of the lowercase text
assembled = assemble(lower,spaceindex);

%Encrypt the assembled text by shifting its ASCII values with a random key
[encrypted,key] = encrypt(assembled);

%Convert the encrypted text into an 8-bit binary matrix, and flip it based
% on whether the key is even or odd
finalresult = permutate(encrypted,key);

%Display the final encrypted message
disp("encrypted message: ");
disp(finalresult);

%Display the random key used for encryption
disp("key: ");
disp(key);
end

%a
%Remove spaces from the text and save their original positions
function [nospacestr, spaceindex] = blankout (plaintext)
    if isstring(plaintext)      %Check if the input is a string
         converted_char = char(plaintext);  %If it's a string, convert it
         % into a character array
    elseif ischar(plaintext) %Or check if the input is already a character
        % array
         converted_char = plaintext; %%Keep the input the same if it's
         % already a character array
    end

    nospacechar = ''; %Create an empty character array to store the text
    % without spaces
    spaceindex = []; %Create an empty vector to save the original positions
```

```
    % of the spaces

    for i = 1:length(converted_char) %Use a for loop to go through each
        % character in the text one at a time
        if isspace(converted_char(i)) %Check if the current character is a
            % space
            spaceindex(end+1) = i; %If it's a space, save its original
            % position at the end of the vector
            continue;%To skip these for loop  so the space won't be added
            % to our new text
        end
        nospacechar(end+1) = converted_char(i); %Add the current character
        % to our new text if it's not a space
    end
    nospacestr = string(nospacechar); %Convert the text without spaces
    % into a MATLAB string
end

%b
%This function puts the saved space positions together with the
% lowercase text
function assembly = assemble(string,index_vector)
    charred_index = num2str(index_vector); %%Convert the space positions
    % from numbers into text so we can combine them with the string
    appended_char = strrep(charred_index,' ','B'); %Use strrep as mentioned
    % in the worksheet to replace the blank spaces between the numbers with
'B'
    assembly = [appended_char string];
end

%c written by May
function [outarg1,outarg2]=lowercase(string)
    outarg1=lower(string); %Use lower() function to convert uppercase
    % letters into lowercase.
    outarg2=find(isstrprop(string,'upper')); %Use isstrprop() function to
    % check which letters were uppercase and find to get their original
    % positions
    %Looked up on Google how to find uppercase letters in a string and
    % learned that isstrprop can check for them, while find gives their
    % positions
end

%d
%This function encrypts the message by shifting each character's ASCII
% value using a randomly generated key
function [encrypted,key] = encrypt(assem)
    googoo = randi([1,127]);  %Generate a random number between 1 and 127
    % to use as our encryption key
    if isstring(assem)  %Check if the input is a string
        char_assem = char(assem); %If it is, convert it into a character
        % array
    elseif ischar(assem)  %Or check if the input is already a character
        % array
        char_assem = assem; %Then keep it the same if it's already a
```

```
        % character array
    end
    shifted = []; %Create an empty vector to store the shifted ASCII values
    for i = 1:length(char_assem) %Use a for loop to go through each
        % character one at a time
        ascii_code = double(char_assem(i)); %Convert the current character
        % into its ASCII value
        shifted(end+1) = ascii_code + googoo + i;

        if shifted >= 127 %Check if the shifted values are greater than or
            % equal to 127
            shifted = shifted - 127; %Subtract 127 if the condition is true
            % to bring the values back down
        end
    end
            encrypted = char(shifted);  %Convert the shifted ASCII values
            % back into characters
            key = googoo; %Save the random number as our key
end

%e.
%This function converts the encrypted message into binary and flips
% the matrix depending on whether the key is even or odd
function outarg=permutate(string,key)
    outarg=dec2bin(string.',8); %Use dec2bin like the example from the
    % worksheet to convert each character into 8-bit binary
    if mod(key,2)==0 %Use mod to check if the key is even by seeing if the
        % remainder is 0
        outarg=flipud(outarg); %If the key is even, flip the entire matrix
        % up-down
    else %Or the key is odd
        outarg=fliplr(outarg); %Flip the entire matrix left-right instead
    end
end

%May Liu worked on Parts c and e and wrote the explanations.
%Jiale Jiang worked on all the rest of the code and helped go through
%the code and comments.
```

Published with MATLAB® R2026b